Data-Driven Optimization

선형모형 모델링 - 고급

6주차

군집화 기반 자원 할당 및 입지 최적화

중간고사 공지

- 일시: 4월 23일 (목) 10시 (100분 예정)
- 장소: 1004-5호 PC 실
- 주의 사항
  - 시험은 모델 작성(필기)과 코딩으로 구성됨.
  - 필기시험을 위해 볼펜 지참
  - 프로그램 코딩을 위한 기본 템플릿 제공 예정(코딩을 외울 필요 없음.)
  - 생성형 AI는 사용 금지. 사용이 의심되는 경우 본인 소명 필요

오늘 수업 내용

1. P-median problem: 전체적인 효율성 (평균)
   - 다수의 행복, 전체적인 시스템 효율성
   - 수요가 많은 곳(인구 밀집 지역)에 시설이 가깝게 배치
   - 대형 마트나 택배 물류센터의 위치 선정. (대다수 고객의 이동 시간을 줄여 전체 비용을 아끼는 목적)
2. P-center problem: 형평성과 최악의 상황 (최대)
   - 소외되는 사람 없는 형평성, 긴급 상황 대응
   - 아무리 외진 곳에 사는 사람이라도 일정 시간 내에 서비스를 받을 수 있도록 보장, 시설이 전체 지역에 골고루 퍼지는 경향이 있음
   - 소방서, 응급실, 경찰서의 위치 선정. (가장 먼 집이라도 골든타임 내에 도착해야 하므로)
3. K-means algorithm for clustering
   - 유사성 → 데이터 속에 숨겨진 그룹 구조
   - 고객 세분화

오늘 수업 내용

1. P-median problem
2. P-center problem
3. K-means algorithm for clustering

1. P-median 문제

- n개의 수요 지점을 p개의 시설에서 서비스할 때, 수요-가중 총 이동 거리를 최소화하는 p개의 입지를 선택하는 문제
- 파라미터
  - w_i: 수요 지점 i의 수요량, $i = 1, \dots, n$
  - d_{ij}: 수요 지점 i와 후보 입지 j 간의 거리, 수요량, $i = 1, \dots, n, j = 1, \dots, m$
  - p: 개설할 시설 수
- 의사결정변수(이진변수)
  - y_j: 후보 입지 j에 시설 개설 여부, $j = 1, \dots, m$
  - x_{ij}: 수요 지점 i 를 시설 j에 할당 여부,$i = 1, \dots, n, j = 1, \dots, m$

$$\min \sum_i \sum_j w_i d_{ij} x_{ij}$$

s. t

$$\sum_j x_{ij} = 1 \quad for \ \forall i$$

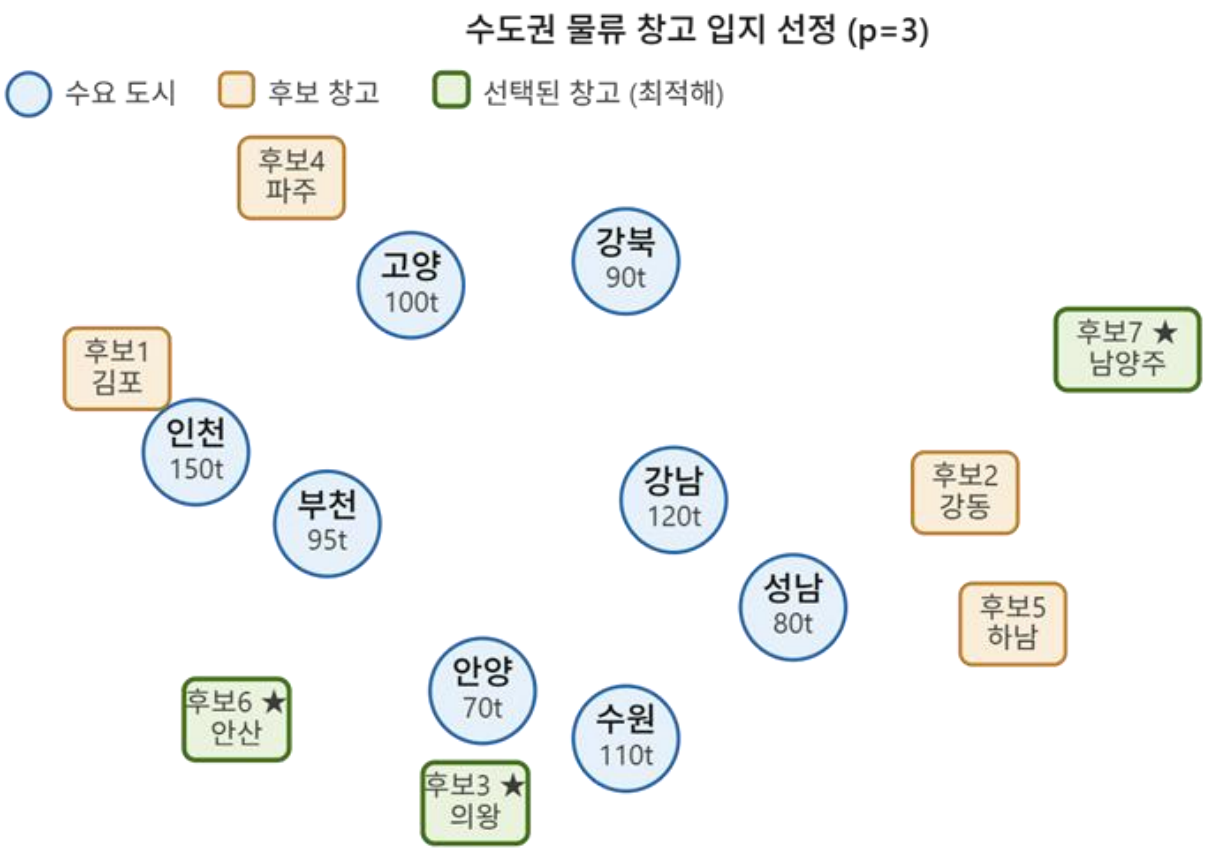
$$x_{ij} \le y_j \quad for \ \forall (i,j)$$

$$\sum_j y_j = p$$

모든 x_{ij}, y_j는 이진변수

1. P-median 문제

	후보1 (김포)	후보2 (강동)	후보3 (의왕)	후보4 (파주)	후보5 (하남)	후보6 (안산)	후보7 (남양주)	수요량
강남	38	15	22	55	20	40	28	120
강북	25	30	40	32	35	55	25	90
인천	18	52	48	45	58	28	62	150
수원	40	38	15	65	42	22	50	110
성남	45	18	16	58	22	32	30	80
고양	20	38	50	18	42	58	38	100
부천	15	38	32	35	45	25	48	95
안양	32	35	12	60	40	18	45	70



1. P-median 문제 - coding

```python
from ortools.sat.python import cp_model

demand_cities = ["강남", "강북", "인천", "수원", "성남", "고양", "부천",
"안양"]
candidates    = ["김포", "강동", "의왕", "파주", "하남", "안산", "남양주"]

# 각 수요 도시의 월간 수요량 (톤)
weights = [120, 90, 150, 110, 80, 100, 95, 70]

# 거리 행렬 d[i][j]: 수요 도시 i → 후보 창고 j (km)
#   김포  강동  의왕  파주  하남  안산  남양주
distances = [
    [38,  15,  22,  55,  20,  40,  28],   # 강남
    [25,  30,  40,  32,  35,  55,  25],   # 강북
    [18,  52,  48,  45,  58,  28,  62],   # 인천
    [40,  38,  15,  65,  42,  22,  50],   # 수원
    [45,  18,  16,  58,  22,  32,  30],   # 성남
    [20,  38,  50,  18,  42,  58,  38],   # 고양
    [15,  38,  32,  35,  45,  25,  48],   # 부천
    [32,  35,  12,  60,  40,  18,  45],   # 안양
]

n = len(demand_cities)   # 수요 도시 수
m = len(candidates)      # 후보 창고 수
p = 3                    # 선택할 창고 수

model  = cp_model.CpModel()
solver = cp_model.CpSolver()

# 결정변수
x = [model.NewBoolVar(f"x_{candidates[j]}") for j in range(m)]
y = [[model.NewBoolVar(f"y_{demand_cities[i]}_{candidates[j]}")
      for j in range(m)] for i in range(n)]
```

```python
# 제약 (1): 정확히 p개 창고 선택
model.Add(sum(x) == p)

# 제약 (2): 각 수요 도시는 정확히 1개 창고에 할당
for i in range(n):
    model.Add(sum(y[i]) == 1)

# 제약 (3): 선택된 창고에만 할당 가능
for i in range(n):
    for j in range(m):
        model.Add(y[i][j] <= x[j])

# 목적함수: 가중 거리 합계 최소화
# CP-SAT는 정수형만 처리하므로 실수 거리를 그대로 정수로 사용 (이미 정수 데이터)
obj_terms = []
for i in range(n):
    for j in range(m):
        obj_terms.append(weights[i] * distances[i][j] * y[i][j])
model.Minimize(sum(obj_terms))

solver.parameters.max_time_in_seconds = 60.0
status = solver.Solve(model)
```

출력 코드

```
if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    tag = "최적해" if status == cp_model.OPTIMAL else "실행가능해"
    print(f"\n  풀이 상태 : {tag}")
    print(f"  목적함수값: {int(solver.ObjectiveValue()):,} (톤·km/월)")

    # 선택된 창고
    selected = [j for j in range(m) if solver.Value(x[j]) == 1]
    print(f"\n  ▶ 선택된 창고 ({p}곳):")
    for j in selected:
        print(f"     - 후보{j+1}: {candidates[j]}")

    # 할당 결과
    print(f"\n  ▶ 수요 도시별 할당 창고:")
    print(f"  {'수요 도시':<8} {'수요량':>7}  {'배정 창고':<10} {'거리':>6}  {'기여 비용':>10}")
    print(f"  {'-'*50}")
    total_cost = 0
    for i in range(n):
        assigned = next(j for j in range(m) if solver.Value(y[i][j]) == 1)
        cost = weights[i] * distances[i][assigned]
        total_cost += cost
        print(f"  {demand_cities[i]:<8} {weights[i]:>5}톤   {candidates[assigned]:<10} "
              f"{distances[i][assigned]:>4}km   {cost:>8,}톤·km")
    print(f"  {'-'*50}")
    print(f"  {'합계':<8} {sum(weights):>5}톤              {'':>4}     {total_cost:>8,}톤·km")

    # 창고별 서비스 권역 요약
    print(f"\n  ▶ 창고별 서비스 권역:")
    for j in selected:
        served = [demand_cities[i] for i in range(n) if solver.Value(y[i][j]) == 1]
        load   = sum(weights[i] for i in range(n) if solver.Value(y[i][j]) == 1)
        print(f"     {candidates[j]}: {', '.join(served)} (총 {load}톤)")

else:
    print("  풀이 실패: 해를 찾지 못했습니다.")

print(f"\n  풀이 시간: {solver.WallTime():.4f}초")
print("=" * 60)
```

2. P-center 문제

- n개의 지점을 p개의 시설에서 서비스할 때, 최대 이동거리(커버 반경)을 최소화하는 문제 (소방서, 구급대처럼 응답시간이 중요할 때 활용됨)
- 파라미터
  - d_{ij}: 지점 i와 후보 입지 j 간의 거리, 수요량, $i = 1, \dots, n, j = 1, \dots, m$
  - p: 개설할 시설 수
- 의사결정변수(이진변수)
  - y_j: 후보 입지 j에 시설 개설 여부, $j = 1, \dots, m$
  - x_{ij}: 수요 지점 i 를 시설 j에 할당 여부,$i = 1, \dots, n, j = 1, \dots, m$
  - W: 최대 커버 거리, 연속 변수 (추가)

$$\min \ W$$

s. t

$$\sum_j d_{ij} x_{ij} \le W \quad for \ \forall i$$

$$\sum_j x_{ij} = 1 \quad for \ \forall i$$

$$x_{ij} \le y_j \quad for \ \forall (i,j)$$

$$\sum_j y_j = p$$

모든 x_{ij}, y_j는 이진변수

$W \ge 0$, 실수

P-center 문제 - coding

```python
from ortools.sat.python import cp_model

# ── 데이터 ──────────────────────────────────────────────────────────


demand_nodes = [
    "수원시", "성남시", "안양시", "부천시",
    "고양시", "남양주시", "안산시", "광명시",
    "평택시", "의정부시"
]

candidates = [
    "수원(후보A)", "성남(후보B)", "안양(후보C)", "부천(후보D)",
    "고양(후보E)", "하남(후보F)", "안산(후보G)", "의정부(후보H)"
]

# 거리 행렬 d[i][j]: 수요지 i → 후보지 j  (분 단위 응급 도달 시간)
#  A수원  B성남  C안양  D부천  E고양  F하남  G안산  H의정부
distances = [
    [ 5,  28,  22,  38,  48,  30,  25,  55],   # 수원시
    [28,   5,  18,  40,  42,  15,  38,  42],   # 성남시
    [22,  18,   5,  30,  40,  25,  20,  50],   # 안양시
    [38,  40,  30,   5,  22,  42,  28,  38],   # 부천시
    [48,  42,  40,  22,   5,  35,  45,  20],   # 고양시
    [30,  15,  25,  42,  35,   5,  42,  30],   # 남양주시
    [25,  38,  20,  28,  45,  42,   5,  58],   # 안산시
    [32,  28,  18,  18,  28,  30,  22,  40],   # 광명시
    [30,  48,  42,  52,  68,  55,  35,  72],   # 평택시
    [55,  42,  50,  38,  20,  30,  58,   5],   # 의정부시
]

n = len(demand_nodes)
m = len(candidates)
p = 3   # 설치할 응급 센터 수
```

```python
model  = cp_model.CpModel()
solver = cp_model.CpSolver()

# 최대 거리값 (상한 계산)
max_dist = max(distances[i][j] for i in range(n) for j in range(m))

# 결정변수
x = [model.NewBoolVar(f"x_{j}") for j in range(m)]
y = [[model.NewBoolVar(f"y_{i}_{j}") for j in range(m)] for i in range(n)]
D = model.NewIntVar(0, max_dist, "D")   # 최대 배정 거리 (최소화 대상)

# 제약 (1): 정확히 p개 센터 선택
model.Add(sum(x) == p)

# 제약 (2): 각 수요지는 정확히 1개 센터에 할당
for i in range(n):
    model.Add(sum(y[i]) == 1)

# 제약 (3): 선택된 센터에만 할당 가능
for i in range(n):
    for j in range(m):
        model.Add(y[i][j] <= x[j])

# 제약 (4): 각 수요지의 배정 거리가 D 이하
# Σ_j d_ij * y_ij ≤ D  →  선택된 j는 y_ij=1 하나뿐이므로 = d_i(assigned)
for i in range(n):
    model.Add(
        sum(distances[i][j] * y[i][j] for j in range(m)) <= D
    )

# 목적함수: D 최소화 (최악 접근 시간 최소화)
model.Minimize(D)

solver.parameters.max_time_in_seconds = 60.0
status = solver.Solve(model)
```

P-center 문제
- 출력 코드

```
print("=" * 65)
print("  P-Center 문제: 경기도 응급 의료센터 최적 배치 결과")
print("=" * 65)

if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    tag = "최적해" if status == cp_model.OPTIMAL else "실행가능해"
    opt_D = int(solver.ObjectiveValue())
    print(f"\n  풀이 상태  : {tag}")
    print(f"  최대 도달 시간 (D*): {opt_D}분  ← 이게 최소화된 커버리지 반경")

    selected = [j for j in range(m) if solver.Value(x[j]) == 1]
    print(f"\n  ▶ 선택된 응급 센터 ({p}곳):")
    for j in selected:
        print(f"     - {candidates[j]}")

    print(f"\n  ▶ 수요지별 할당 결과:")
    print(f"  {'수요지':<12} {'배정 센터':<16} {'도달 시간':>8}  {'최악?':>5}")
    print(f"  {'-'*50}")
    for i in range(n):
        assigned = next(j for j in range(m) if solver.Value(y[i][j]) == 1)
        dist = distances[i][assigned]
        worst = "★ 최악" if dist == opt_D else ""
        print(f"  {demand_nodes[i]:<12} {candidates[assigned]:<16} {dist:>5}분  {worst}")
    print(f"  {'-'*50}")
    print(f"  {'최대 도달 시간':<28} {opt_D:>5}분")

    print(f"\n  ▶ 센터별 서비스 권역:")
    for j in selected:
        served = [demand_nodes[i] for i in range(n) if solver.Value(y[i][j]) == 1]
        max_d  = max(distances[i][j] for i in range(n) if solver.Value(y[i][j]) == 1)
        print(f"     {candidates[j]}: {', '.join(served)} (최대 {max_d}분)")

else:
    print("  풀이 실패: 해를 찾지 못했습니다.")

print(f"\n  풀이 시간: {solver.WallTime():.4f}초")
print("=" * 65)
```

3. K-means clustering

- K-Means 클러스터링은 주어진 데이터를 K개의 그룹(클러스터)으로 나누는 비지도 학습(Unsupervised Learning) 알고리즘
  - 고객 세그먼테이션: 소비 패턴에 따라 구매 고객을 그룹화하여 타겟 마케팅 수행
  - 이미지 압축: 이미지의 수만 가지 색상을 K개의 대표 색상으로 묶어 용량 최적화
  - 이상치 탐지: 어떤 클러스터에도 잘 속하지 않거나 중심에서 너무 먼 데이터를 불량품/사기로 간주.
- 알고리즘
  - 데이터 중 임의의 점 K개를 선택하여 중심점(Centroid)으로 선정
  - 모든 데이터를 가장 가까운 중심점에 할당
  - 할당된 데이터들의 평균 위치로 중심점을 이동
  - 중심점이 변하지 않을 때까지 2~3단계를 반복

3. K-means clustering

- 목적함수: 모든 점에서 자신이 속한 클러스터의 중심까지 거리의 합을 최소화 (오차 제곱합, SSE)

$$\sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

- k: 클러스터 개수
- C_i: i번째 클러스터에 속하는 데이터 집합
- μ_i: i번째 클러스터의 중심점

3. K-means clustering

6-3.py

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# 1. 가상 데이터 생성
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# 2. K-Means 모델 생성 및 학습 (K=4)
kmeans = KMeans(n_clusters=5)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)

# 3. 결과 시각화
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.5,
marker='X')
plt.title("K-Means Clustering Result")
plt.show()
```

```python
# 2. K-Means 모델 생성 및 학습 (K=4)
def get_distance(p1, p2):
    """두 점 사이의 유클리드 거리를 계산 (루트 없이 제곱합으로만 비교해도 무방함)"""
    return math.sqrt(sum((a - b) ** 2 for a, b in zip(p1, p2)))

# 설정값
k = 4
max_iters = 100
n_samples = len(X)
n_features = len(X[0])

# 1. 초기 중심점 설정: 데이터 중 랜덤하게 k개 선택
# (X가 numpy array라면 X.tolist()로 변환해서 사용 가능)
data_list = X.tolist()
centroids = random.sample(data_list, k)
```

6-4.py

```python
for iteration in range(max_iters):
    # 2. 할당(Assignment): 각 데이터를 가장 가까운 중심점 그룹에 번호로 기록
    clusters = []
    for point in data_list:
        distances = [get_distance(point, c) for c in centroids]
        cluster_idx = distances.index(min(distances))
        clusters.append(cluster_idx)

    # 3. 업데이트(Update): 각 클러스터의 평균을 구해 새로운 중심점 계산
    new_centroids = []
    for i in range(k):
        # i번째 클러스터에 속한 점들만 모으기
        group_points = [data_list[j] for j in range(n_samples) if clusters[j] == i]

        if group_points:
            # 각 차원별(x, y)로 합계를 구해 평균 계산
            new_centroid = [sum(dim) / len(group_points) for dim in zip(*group_points)]
            new_centroids.append(new_centroid)
        else:
            # 만약 클러스터에 데이터가 하나도 없다면 기존 중심점 유지
            new_centroids.append(centroids[i])

    # 4. 수렴 확인: 중심점이 변하지 않으면 중단
    if new_centroids == centroids:
        print(f"{iteration}회 반복 후 최적의 중심점에 도출했습니다.")
        break
    centroids = new_centroids

# 시각화를 위해 결과값을 다시 형식에 맞춰줌
y_kmeans = clusters
centers = np.array(centroids)
```

3. 최적 K 결정 – 엘보우 방법

- K값 변화: K를 1부터 10(또는 적절한 수)까지 1씩 늘려가며 K-Means 알고리즘을 각각 실행
- SSE 계산: 각 K값에 대해 학습이 완료된 후의 SSE 값을 계산
- 그래프 그리기: X축을 K값, Y축을 SSE 값으로 두고 꺾은선 그래프를 그림
- 팔꿈치 찾기: 그래프의 모양이 팔이 꺾이는 형태(팔꿈치)처럼 경사가 급격히 완만해지는 지점의 K가 최적의 클러스터 개수

- K가 커지면 커질수록 당연히 SSE는 작아짐. 따라서, 클러스터 수르르 늘려서 얻는 이익이 급격히 줄어드는 한계점을 찾는 것임.

3. 최적 K 결정 – 엘보우 방법

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# 1. 가상 데이터 생성
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# scikit-learn을 활용한 엘보우 방법 예시
sse = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=0)
    kmeans.fit(X)
    sse.append(kmeans.inertia_) # 각 K마다의 SSE 값을 리스트에 저장

# 그래프 그리기
plt.plot(K_range, sse, marker='o')
plt.xlabel('Number of clusters (K)')
plt.ylabel('SSE (Inertia)')
plt.title('The Elbow Method')
plt.show()\
```

실습